BIO312- Bioinformatics Analysis

ASSIGNMENT

Instructor: Dr. Muhammad Sajjad

Student: Tooba Mir

Assignment: Single-Cell RNA-Seq Data Analysis

Objective:

Analyze single-cell RNA sequencing data to identify cell types & differentially expressed genes.

Tasks:

1. Download a single-cell RNA-seq dataset (e.g., from the 10x Genomics website or GEO database).
2. Preprocess the data using tools like Seurat or Scanpy.
3. Perform dimensionality reduction (e.g., PCA, t-SNE, or UMAP) and clustering to identify cell types.
4. Identify differentially expressed genes between clusters.
5. Use a machine learning model (e.g., k-Nearest Neighbors or SVM) to classify cell types based on gene expression.

Deliverables:

- A report detailing the analysis pipeline, clustering results, and differentially expressed genes.
- A Jupyter notebook or R script with the code for data preprocessing, clustering, and classification.

Single-Cell RNA-Seq Data Analysis Report

Introduction

RNA sequencing (RNA-seq) is a genomic approach for the detection and quantitative analysis of messenger RNA molecules in a biological sample and is useful for studying cellular responses. RNA-seq has fueled much discovery and innovation in medicine over recent years. For practical reasons, the technique is usually conducted on samples comprising thousands to millions of cells. However, this has hindered direct assessment of the fundamental unit of biology—the cell. Since the first single-cell RNA-sequencing (scRNA-seq) study was published in 2009, many more have been conducted, mostly by specialist laboratories with unique skills in wet-lab single-cell

genomics, bioinformatics, and computation. However, with the increasing commercial availability of scRNA-seq platforms and the rapid ongoing maturation of bioinformatics approaches, a point has been reached where any biomedical researcher or clinician can use scRNA-seq data to make exciting discoveries.

Objective

In this assignment, the objective is to analyze single-cell RNA-seq data to identify cell types and differentially expressed genes using tools such as Seurat in R. The process includes downloading a public dataset, preprocessing it, performing dimensionality reduction and clustering, and finally applying a machine learning model for classification.

Step 1: Dataset Selection

For this analysis, I selected the publicly available dataset: "1k Peripheral Blood Mononuclear Cells (PBMCs) from a Healthy Donor (v2 chemistry)" from 10x Genomics. This dataset is widely used in tutorials and benchmarking and is well-suited for beginners in scRNA-seq analysis.

Reasons for choosing this dataset:

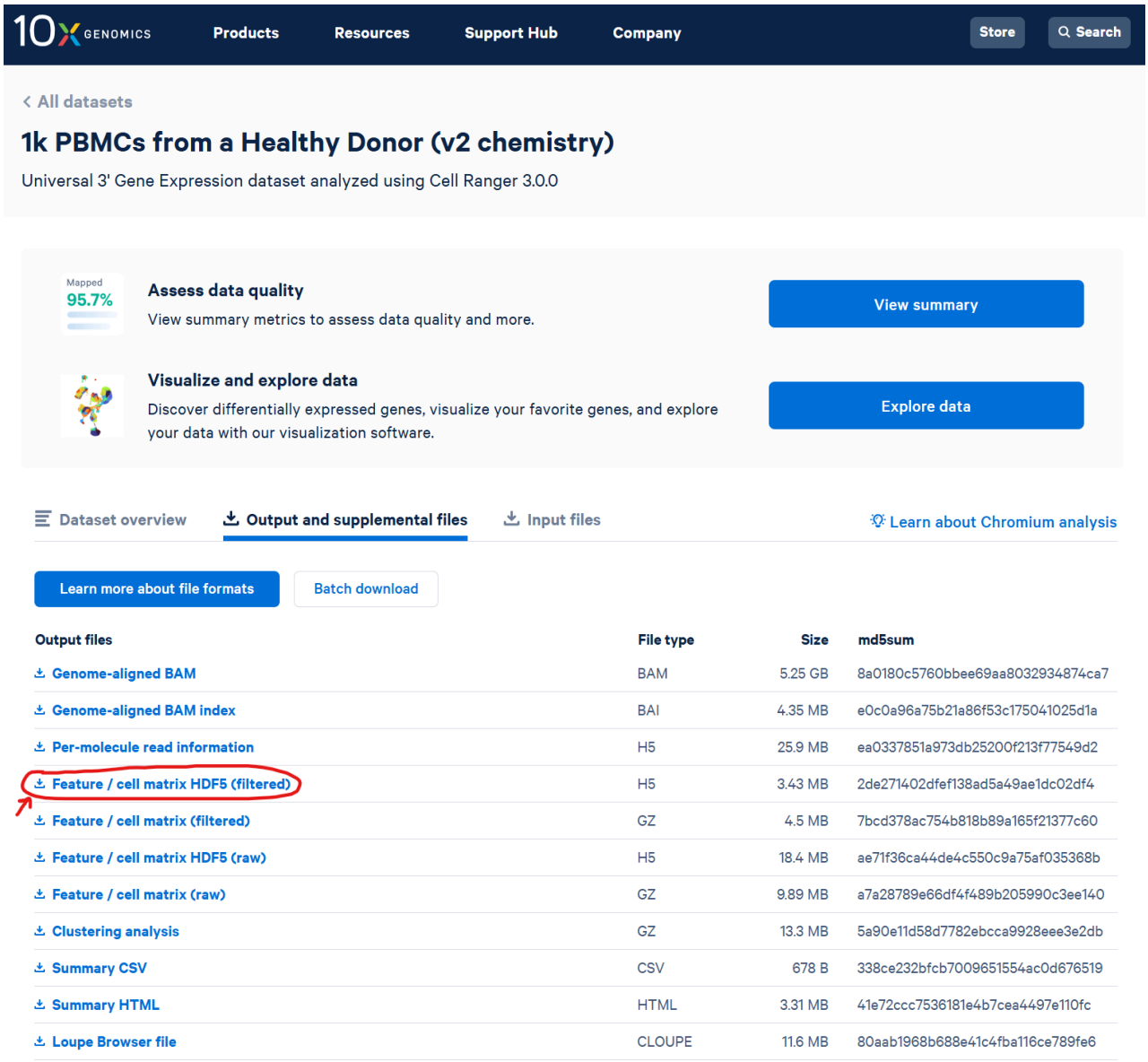
- It contains a well-mixed population of immune cells (T cells, B cells, NK cells, monocytes).
- It has high-quality annotations and a manageable size (~1,000 cells).
- The v2 chemistry version is supported by most tools and examples.

Dataset source: [1k PBMCs from a Healthy Donor (v2 chemistry) - 10x Genomics]

Step 2: Downloading the Data

From the dataset page, I had the option to download either the `filtered_feature_bc_matrix` format (which includes `.tsv` and `.mtx` files) or the HDF5 (.h5) file, which is more compact and easier to load into R.

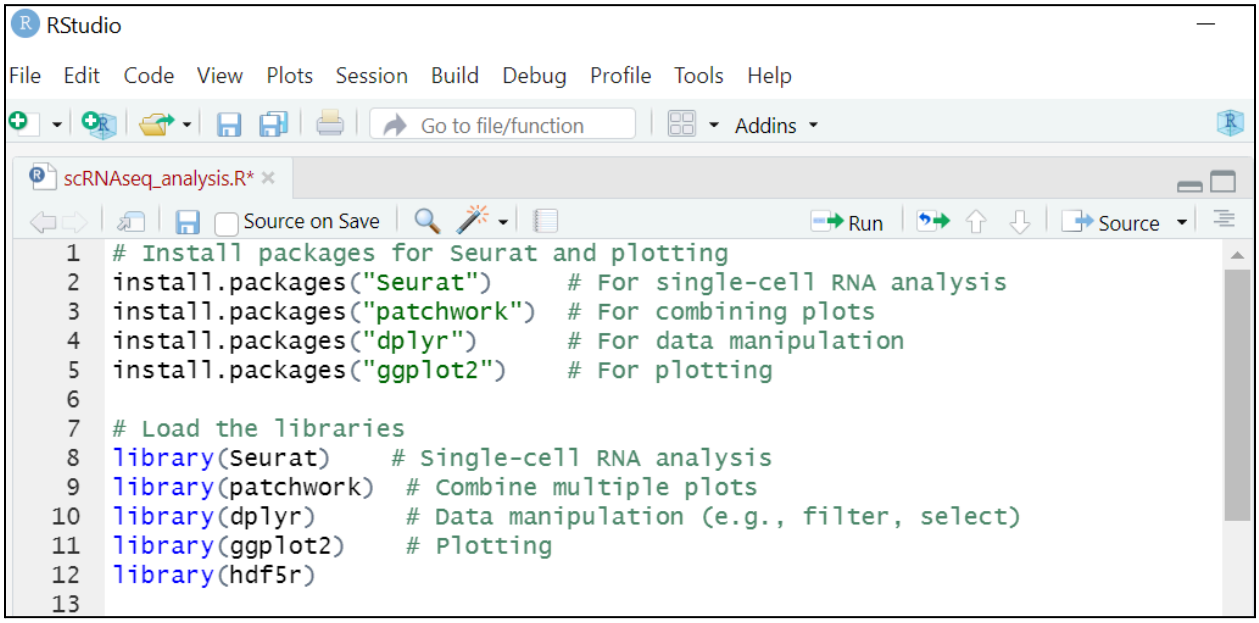
I chose to download the **filtered gene-barcode matrix in HDF5 format**, as shown below:



Step 3: Setting Up R for Analysis

1. **New R Script:** After setting up my working directory and extracting the dataset, I launched RStudio, a user-friendly interface for writing and running R code. Since I had never used R before, I began by creating a new R Script, which allowed me to save each line of code I wrote and execute commands one step at a time.

2. **Installed and loaded the Seurat package**: The Seurat package is one of the most widely used toolkits for single-cell RNA sequencing analysis in R. This package includes functions to load data, perform quality control, normalize gene expression, identify cell clusters, and visualize the results. I installed it by running the following command in my script:



3. **Defined the file path to the .h5 file:**

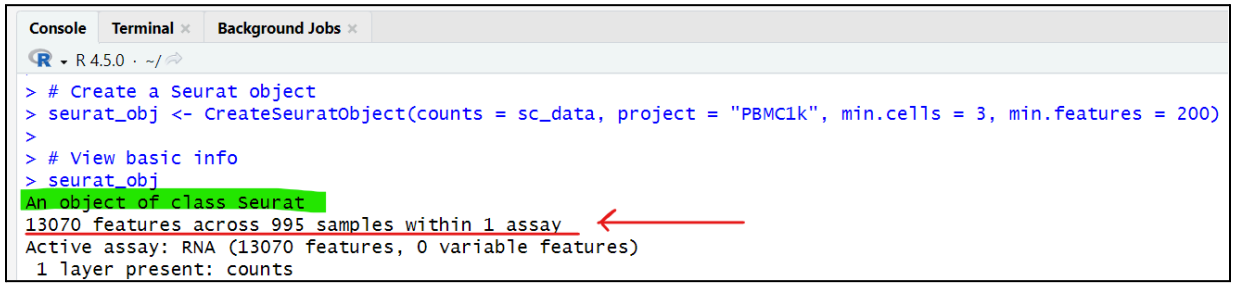
```
14 # path to file
15 h5_file <- "C:/Users/tooba/Downloads/pbmc_1k_v2_filtered_feature_bc_matrix.h5"
16
```

4. **Read the data into R using Seurat's `Read10X_h5()` function:** With the path set, I used Seurat's built-in `Read10X_h5()` function to load the contents of the .h5 file. This

function automatically reads the gene expression matrix along with barcodes (representing individual cells) and features (genes) in a format that Seurat can use:

```
# Load it using Seurat
sc_data <- Read10X_h5(h5_file)

# Create a Seurat object
seurat_obj <- CreateSeuratObject(counts = sc_data, project = "PBMC1k", min.cells = 3, min.features = 200)

# View basic info
seurat_obj
```

5. **Result:** In the console, I can see that I have successfully created a "Seurat object", which means that the single-cell RNA-seq dataset has successfully loaded into R from the .h5 file and is now ready for preprocessing with Seurat.



What does this output mean:

- **13070 features** = ~13,070 genes
- **995 samples** = 995 individual cells
- **1 assay: RNA** = working with RNA expression data
- **0 variable features** = I haven't selected highly variable genes yet

Step 4: Preprocessing the Data using Seurat

1. Quality Control (QC Filtering)

This step filters out bad-quality cells like:

- Cells expressing too few genes (maybe dead)
- Cells expressing too many genes (maybe doublets)
- Cells with high mitochondrial gene % (maybe stressed/dying)

```r
# Calculate % mitochondrial gene content
seurat_obj[["percent.mt"]] <- PercentageFeatureSet(seurat_obj, pattern = "^MT-")

# View QC metrics
VlnPlot(seurat_obj, features = c("nFeature_RNA", "nCount_RNA", "percent.mt"), ncol = 3)
```

This gives violin plots. Violin plots show **how much RNA or how many genes each cell has**, like a chart of cell quality.

For PBMCs (blood cells), we usually **keep only the "healthy-looking" cells**, using these simple rules:

- ***nFeature_RNA* > 200** → Cell must have more than 200 genes (to avoid empty/dying cells)
- ***nFeature_RNA* < 2500** → Less than 2500 genes (too many could mean two cells stuck together)
- ***percent.mt* < 5** → Less than 5% of RNA from mitochondria (too much means stressed/dying cells)

Now filter:

```r
# Filter out low-quality cells
seurat_obj <- subset(seurat_obj, subset = nFeature_RNA > 200 & nFeature_RNA < 2500 & percent.mt < 5)
```

2. Normalization:

This step ensures that differences in gene expression are biologically meaningful, not just due to sequencing depth.

```r
#Normalization
seurat_obj <- NormalizeData(seurat_obj, normalization.method = "LogNormalize", scale.factor = 10000)
```

```
Console   Terminal   Background Jobs
R 4.5.0 · ~/
> seurat_obj <- NormalizeData(seurat_obj, normalization.method = "LogNormalize", scale.factor = 10000)
Normalizing layer: counts
Performing log-normalization
0%   10   20   30   40   50   60   70   80   90   100%
[----|----|----|----|----|----|----|----|----|----|
**************************************************|
```

Importance of Normalization is that it adjusts for how much total RNA each cell has, because some cells might have more RNA just by chance (not because they're biologically different). If we don't normalize, we might think a gene is more active just

because the cell has more total RNA, not because that gene is truly important.

So, normalization helps us compare cells fairly.

3. Find High Quality Variable Genes:

We now pick genes that vary most between cells (informative for clustering).

```
39  seurat_obj <- FindVariableFeatures(seurat_obj, selection.method = "vst", nfeatures = 2000)
```

```
Console  Terminal  Background Jobs
R 4.5.0 · ~/
> seurat_obj <- FindVariableFeatures(seurat_obj, selection.method = "vst", nfeatures = 2000)
Finding variable features for layer counts
Calculating gene variances
0%   10   20   30   40   50   60   70   80   90   100%
[----|----|----|----|----|----|----|----|----|----|
**************************************************|
Calculating feature variances of standardized and clipped values
0%   10   20   30   40   50   60   70   80   90   100%
[----|----|----|----|----|----|----|----|----|----|
**************************************************|
>
```

Optional step: see the top ten variable genes

```
41  # Optional: See top 10 variable genes
42  top10 <- head(VariableFeatures(seurat_obj), 10)
43  top10
```

```
Console  Terminal  Background Jobs
R 4.5.0 · ~/
> # Optional: See top 10 variable genes
> top10 <- head(VariableFeatures(seurat_obj), 10)
> top10
 [1] "S100A8"  "S100A9"  "GNLY"    "IGLC2"   "LYZ"     "IGKC"
 [7] "S100A12" "HLA-DRA" "GZMB"    "FTL"
```

4. Scaling the Data

This makes all genes comparable and helps with PCA later.

```
45  all.genes <- rownames(seurat_obj)
46  seurat_obj <- ScaleData(seurat_obj, features = all.genes)
```

5. **Result:**

The preprocessing pipeline successfully cleaned and standardized the dataset, ensuring that only high-quality, informative cells and genes were retained. This prepared the data for accurate dimensionality reduction, clustering, and identification of distinct cell types in the following analysis steps.

Summary of Preprocessing Steps

Step	Purpose
QC Filtering	Remove low-quality or noisy cells
Normalization	Adjust for sequencing depth
Variable Feature Selection	Focus on informative genes
Scaling	Standardize gene values for PCA

Step 5: Dimensionality Reduction and Clustering to Identify Cell Types.

1. Run PCA (Principal Component Analysis)

This reduces thousands of gene features to key "components" that explain the most variance.

```
seurat_obj <- RunPCA(seurat_obj, features = VariableFeatures(object = seurat_obj))

# View PCA results
print(seurat_obj[["pca"]], dims = 1:5, nfeatures = 5)

# Visualize PCA
VizDimLoadings(seurat_obj, dims = 1, reduction = "pca")
DimPlot(seurat_obj, reduction = "pca")
```

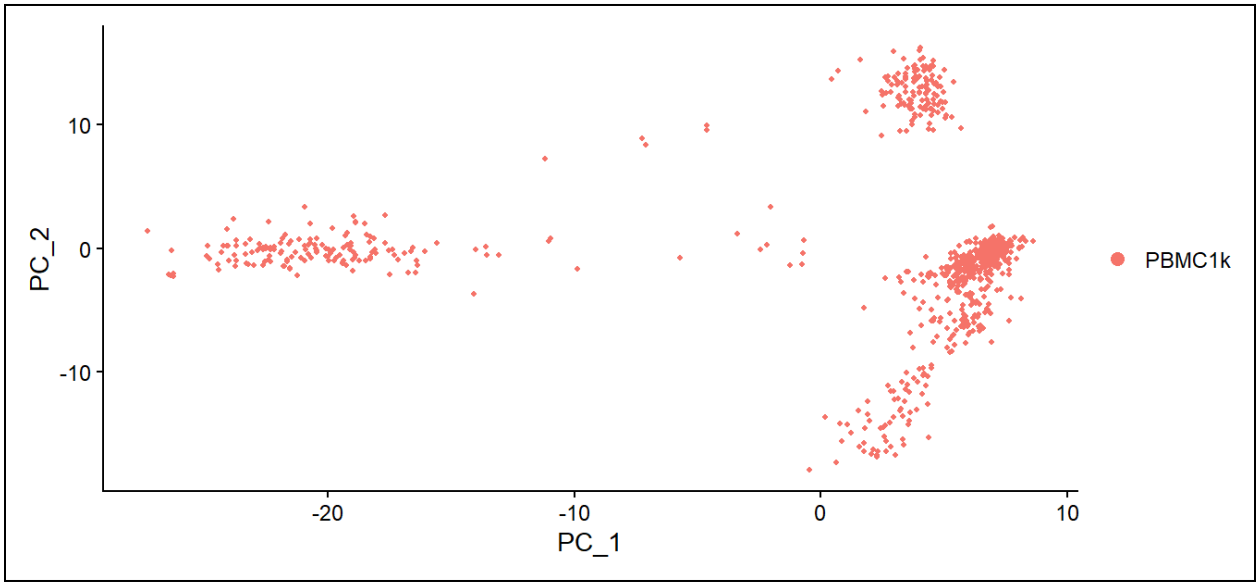
Console:

```
> # View PCA results
> print(seurat_obj[["pca"]], dims = 1:5, nfeatures = 5)
PC_ 1
Positive:  RPS29, LTB, TRAC, CD3D, IL32
Negative:  CST3, CSTA, LYZ, FCN1, LST1
PC_ 2
Positive:  CD79A, IGHM, MS4A1, CD79B, HLA-DQB1
Negative:  NKG7, GZMA, CTSW, PRF1, HCST
PC_ 3
Positive:  EEF1A1, RPS12, RPS18, RPS29, MALAT1
Negative:  PPBP, CAVIN2, GNG11, PF4, TUBB1
PC_ 4
Positive:  GNLY, GZMB, KLRF1, SPON2, KLRD1
Negative:  PF4, GNG11, TUBB1, CAVIN2, PPBP
PC_ 5
Positive:  S100A12, VCAN, AC020656.1, S100A8, CD79A
Negative:  SCT, TNFRSF21, LRRC26, LILRA4, TPM2
>
> # Visualize PCA
> VizDimLoadings(seurat_obj, dims = 1, reduction = "pca")
> DimPlot(seurat_obj, reduction = "pca")
> seurat_obj <- ScaleData(seurat_obj, features = all.genes)
```

Under each PC:

- **Positive genes** = Genes that are strongly expressed in cells on the positive side of this PC
- **Negative genes** = Genes that are strongly expressed on the negative side

Plot:



This is a **PCA plot**, which is a **scatter plot** where:

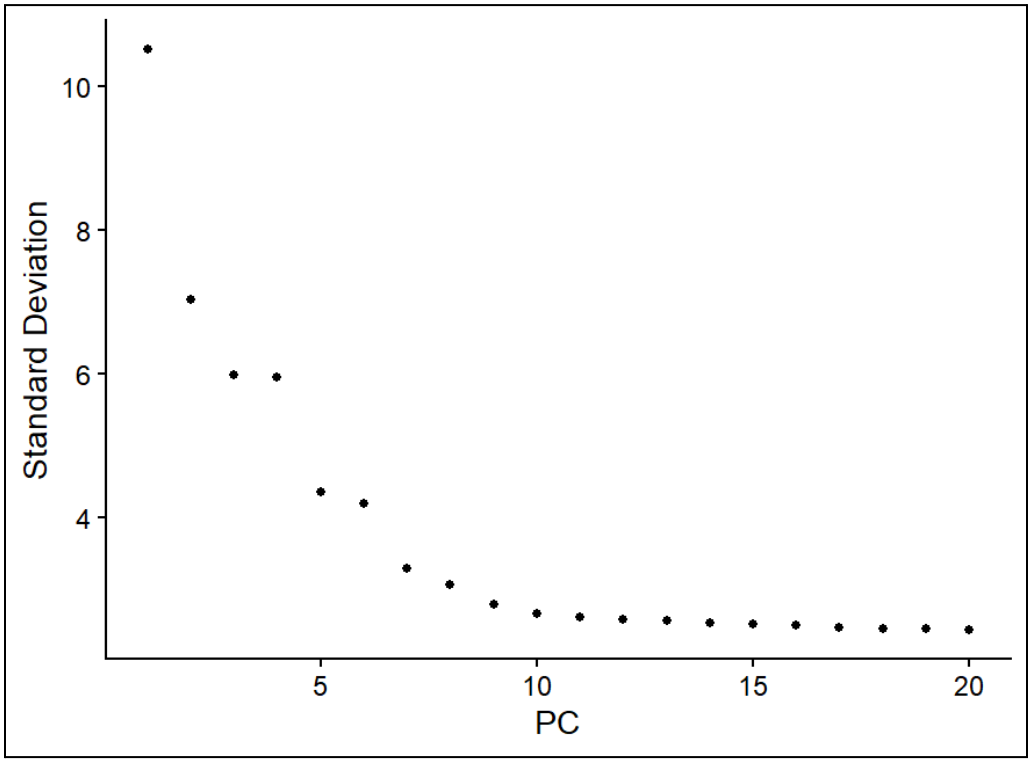
- **Each dot = one cell** from the dataset
- The plot uses **two principal components** (PC1 on the x-axis and PC2 on the y-axis). The position of each cell is based on its overall gene expression pattern, reduced into those two main components
- **Closer dots** = cells with more similar gene expression
- **Further apart dots** = cells with very different expression

At this stage, I am not seeing clusters *yet* — just how cells naturally spread based on the top sources of variation

2. Decide How Many PCs to Use

Use an elbow plot to decide how many principal components (PCs) to keep:

```
58  ElbowPlot(seurat_obj)
```

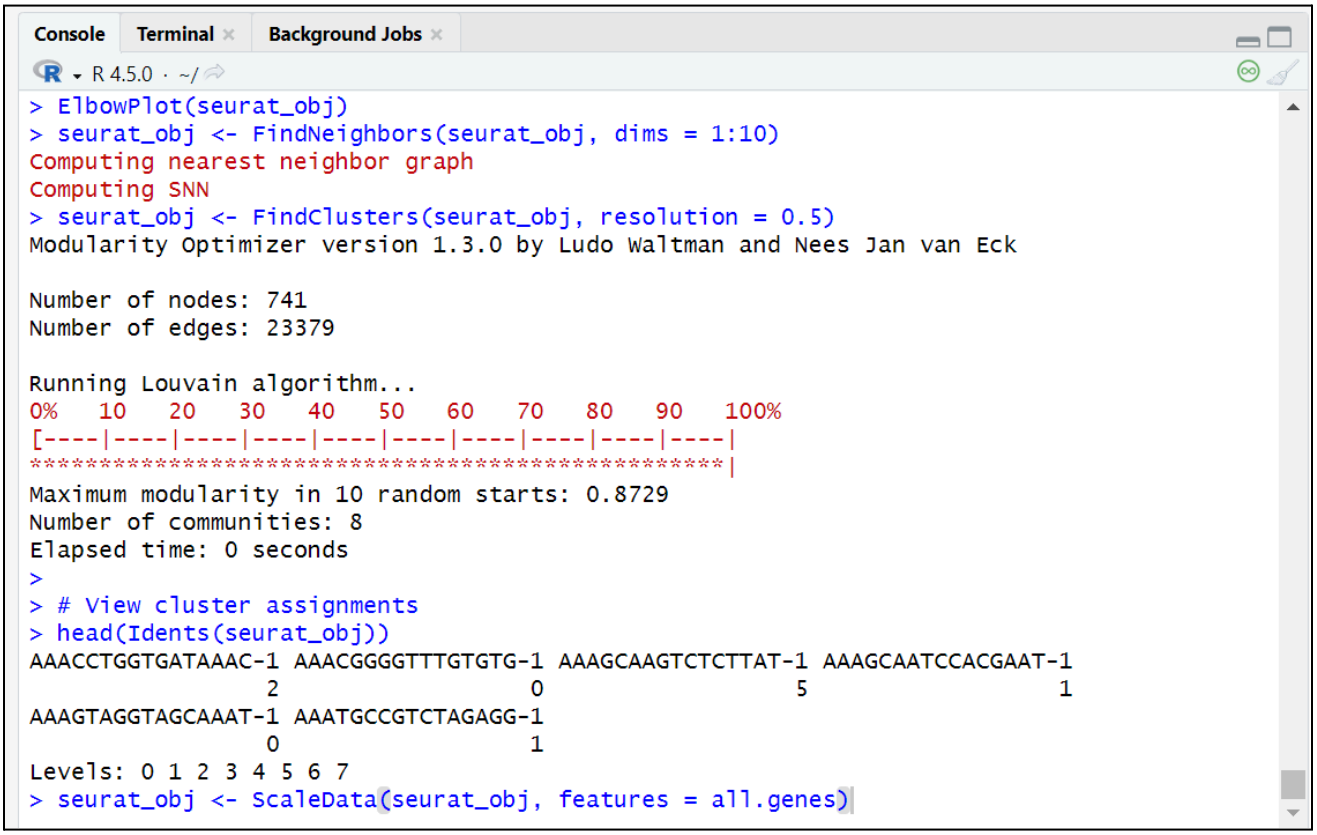


Look for a "bend" (elbow) in the curve. Usually, between **10–20 PCs** are enough. Let's go with 10 for now.

3. Find Neighbors + Cluster the Cells

Now we use those PCs to group similar cells:

```
60  seurat_obj <- FindNeighbors(seurat_obj, dims = 1:10)
61  seurat_obj <- FindClusters(seurat_obj, resolution = 0.5)
62
63  # View cluster assignments
64  head(Idents(seurat_obj))
```



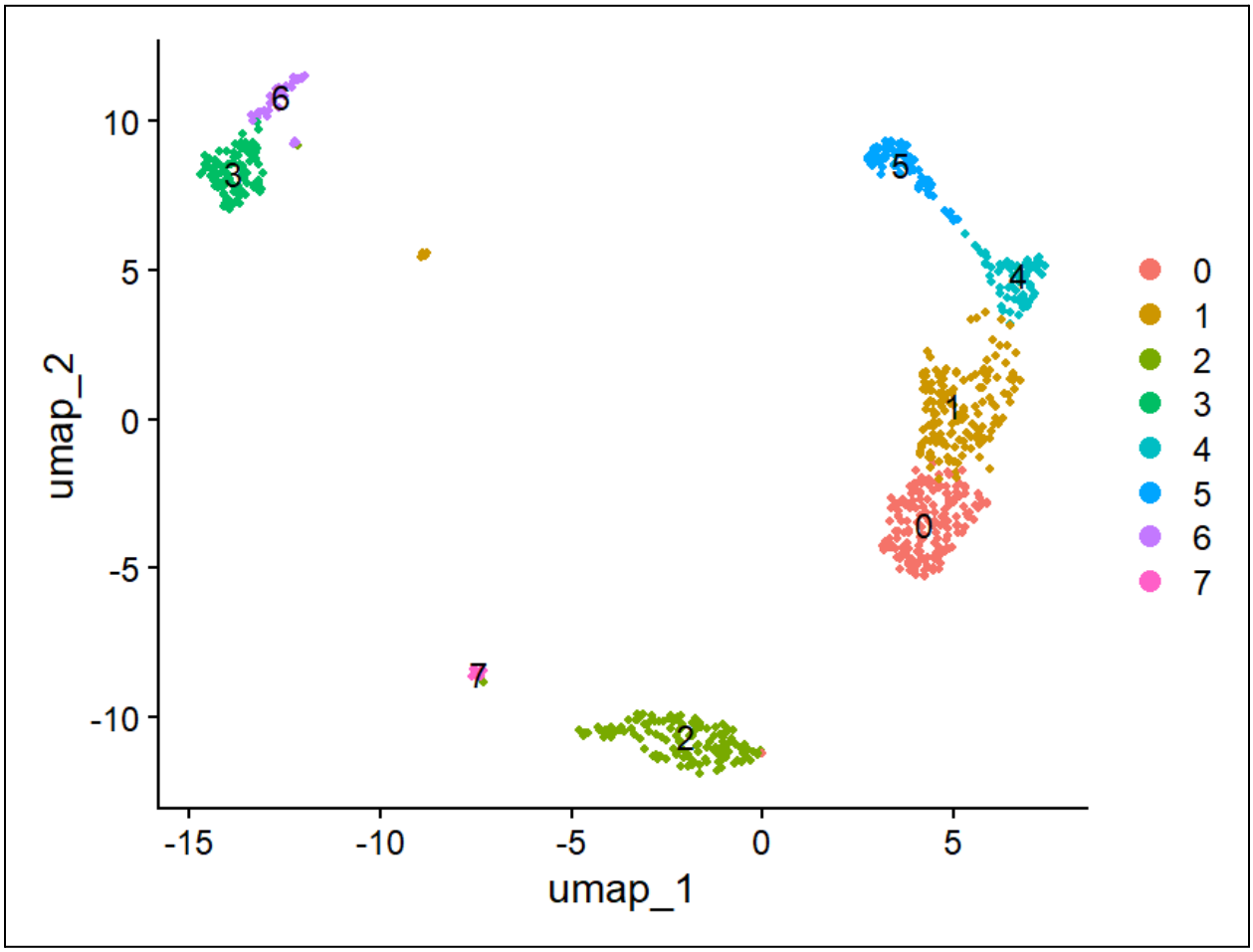
The `resolution` parameter controls how many clusters are found. Try 0.5 first; increase for more clusters.

4. Visualize with UMAP

UMAP (Uniform Manifold Approximation and Projection) makes it easy to **see clusters** in 2D.

```
# View cluster assignments
head(Idents(seurat_obj))

seurat_obj <- RunUMAP(seurat_obj, dims = 1:10)
DimPlot(seurat_obj, reduction = "umap", label = TRUE)
```

Can now see a beautiful UMAP plot with distinct colored clusters



This dataset likely contains 8 distinct cell populations (e.g., T cells, B cells, monocytes, etc.).

Conclusion: The purpose of this step was to simplify and explore the complex single-cell RNA-seq data by reducing it to key features using **PCA**, grouping similar cells through **clustering**, and visualizing them in 2D space using **UMAP**.

From this step, I successfully identified **distinct cell populations** based on their gene expression patterns. Each cluster likely represents a **different cell type or state**, and the UMAP plot provides a visual map of how cells relate to one another in the dataset.

Step 6: Identify Differentially Expressed Genes (DEGs) Between Clusters

This helps find **which genes define each cluster**, giving clues about cell types (e.g., T cells, B cells, etc)

1. Find Marker Genes for Each Cluster

This compares gene expression **between clusters** to find genes that are **highly expressed in one cluster** compared to others.

```
69  cluster_markers <- FindAllMarkers(seurat_obj, only.pos = TRUE, min.pct = 0.25, logfc.threshold = 0.25)
70
71  # View top markers
72  head(cluster_markers)
```

```
Console   Terminal ×   Background Jobs ×
R 4.5.0 · ~/
> # View top markers
> head(cluster_markers)
             p_val avg_log2FC pct.1 pct.2    p_val_adj cluster  gene
RPL21 2.444255e-56  0.6657238     1 0.993 3.194641e-52       0 RPL21
RPS6  1.315201e-54  0.7988141     1 0.986 1.718968e-50       0  RPS6
RPS14 3.551128e-51  0.7018497     1 0.991 4.641325e-47       0 RPS14
RPL32 6.560376e-51  0.6499479     1 0.991 8.574412e-47       0 RPL32
RPS29 1.211345e-49  0.7583937     1 0.991 1.583228e-45       0 RPS29
RPS27 1.863067e-49  0.6761051     1 0.995 2.435028e-45       0 RPS27
>
```

- `only.pos = TRUE`: only keeps upregulated (positive) genes
- `min.pct = 0.25`: keeps genes expressed in at least 25% of cells in either cluster
- `logfc.threshold = 0.25`: only includes genes with a log fold-change > 0.25

2. Check Top Genes for Each Cluster

Can view the top markers (differential genes) per cluster:

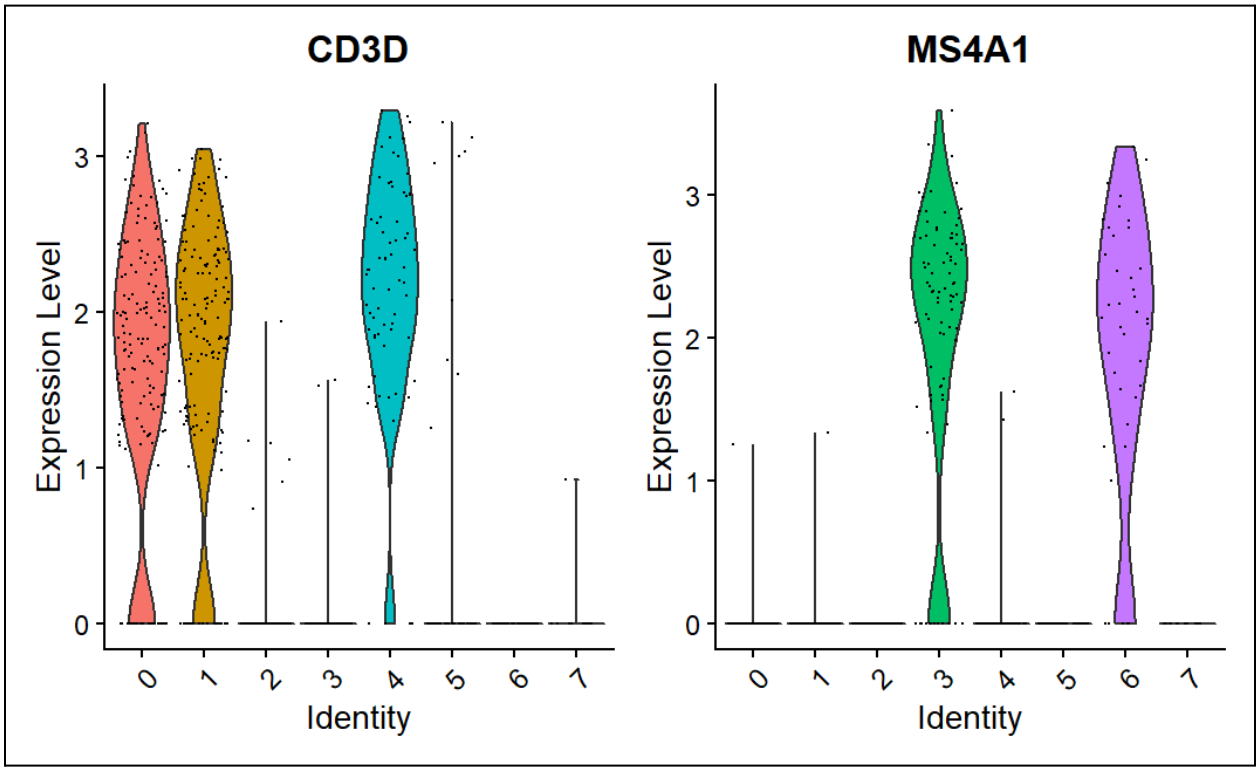
```
top_markers <- cluster_markers %>% group_by(cluster) %>% top_n(n = 3, wt = avg_log2FC)
top_markers
```

```
# A tibble: 24 × 7
# Groups:   cluster [8]
       p_val avg_log2FC pct.1 pct.2 p_val_adj cluster gene
       <dbl>      <dbl> <dbl> <dbl>     <dbl> <fct>   <chr>
 1 1.01e- 40       2.95 0.545 0.092 1.32e- 36 0       LEF1
 2 3.57e- 30       2.22 0.551 0.132 4.67e- 26 0       CCR7
 3 2.86e- 19       2.41 0.341 0.075 3.73e- 15 0       TRABD2A
 4 1.01e- 24       2.53 0.588 0.205 1.32e- 20 1       ITGB1
 5 2.80e- 15       2.03 0.344 0.093 3.66e- 11 1       AQP3
 6 4.60e- 12       1.89 0.281 0.077 6.01e-  8 1       CD5
 7 4.60e-138       9.22 0.896 0.005 6.02e-134 2       VCAN
 8 6.21e- 79       8.94 0.537 0.002 8.12e- 75 2       MGST1
 9 5.44e- 38       9.08 0.261 0     7.10e- 34 2       CREB5
10 6.30e-131       7.24 0.911 0.015 8.23e-127 3       TCL1A
# i 14 more rows
# i Use `print(n = ...)` to see more rows
```

3. Visualize Expression of Marker Genes

Violin plots (show distribution per cluster):

```
VlnPlot(seurat_obj, features = c("CD3D", "MS4A1"))  # example markers for T & B cells
```



What the Plot Shows

Each vertical violin represents the expression of a gene in a cluster (labeled 0–7).

- Y-axis = Expression Level of the gene (normalized)
- X-axis = Clusters (the Seurat-defined clusters)
- The shape of each violin shows the distribution of gene expression values across cells in that cluster.
- The dots are individual cells.

CD3D (left plot)

- Highly expressed in clusters 0, 1, and 4
- Almost not expressed in clusters 2, 3, 5, 6, and 7
- CD3D is a classic T-cell marker
    - So: Clusters 0, 1, and 4 likely represent T cells

MS4A1 (right plot)

- Highly expressed in clusters 3 and 6
- Almost not expressed elsewhere
- MS4A1 is another name for CD20, a B-cell marker
    - So: Clusters 3 and 6 likely represent B cells

4. Heatmap of top genes:

```
79  DoHeatmap(seurat_obj, features = top_markers$gene) + NoLegend()
```

Use these marker genes to annotate the clusters (e.g., "Cluster 0 = T cells")

This **heatmap** shows the expression of selected **marker genes** across different **cell clusters (0–7)** from single-cell RNA sequencing data.

What it represents:

- **Rows** = Genes (e.g., *LEF1*, *CCR7*, *CD5*, *IGHG1*, etc.)
- **Columns** = Clusters (colored bars at top: 0–7)
- **Colors**:
    - **Yellow** = High expression
    - **Pink/Purple** = Low or no expression

Interpretation (based on gene markers):

- **Clusters 0 & 1**: High expression of *LEF1*, *CCR7*, *TRABD2A* → likely **naive/central memory T cells**
- **Cluster 2**: High *VCAN*, *MGST1*, *CREB5* → likely **monocytes** or **inflammatory cells**
- **Cluster 3**: High *TCL1A*, *IGHD* → **naive B cells**
- **Cluster 4**: High *FCER2* → **memory B cells**
- **Cluster 5**: High *IGHG1*, *IGHG3*, *IGHA1* → **plasma cells**
- **Cluster 6**: Moderate *FCER2*, *IGH genes* → possibly **late B cell/plasma-like**
- **Cluster 7**: Low overall expression but may include **rare or transitional cells**

Step 7: Using a Machine Learning Model to Classify Cell Types Based on Gene Expression

Now we'll use a machine learning model (like k-Nearest Neighbors) to classify cells based on gene expression.

Required Installation

```
install.packages("caret")
install.packages("e1071")
install.packages("randomForest")
```

1. Prepare the Data for ML

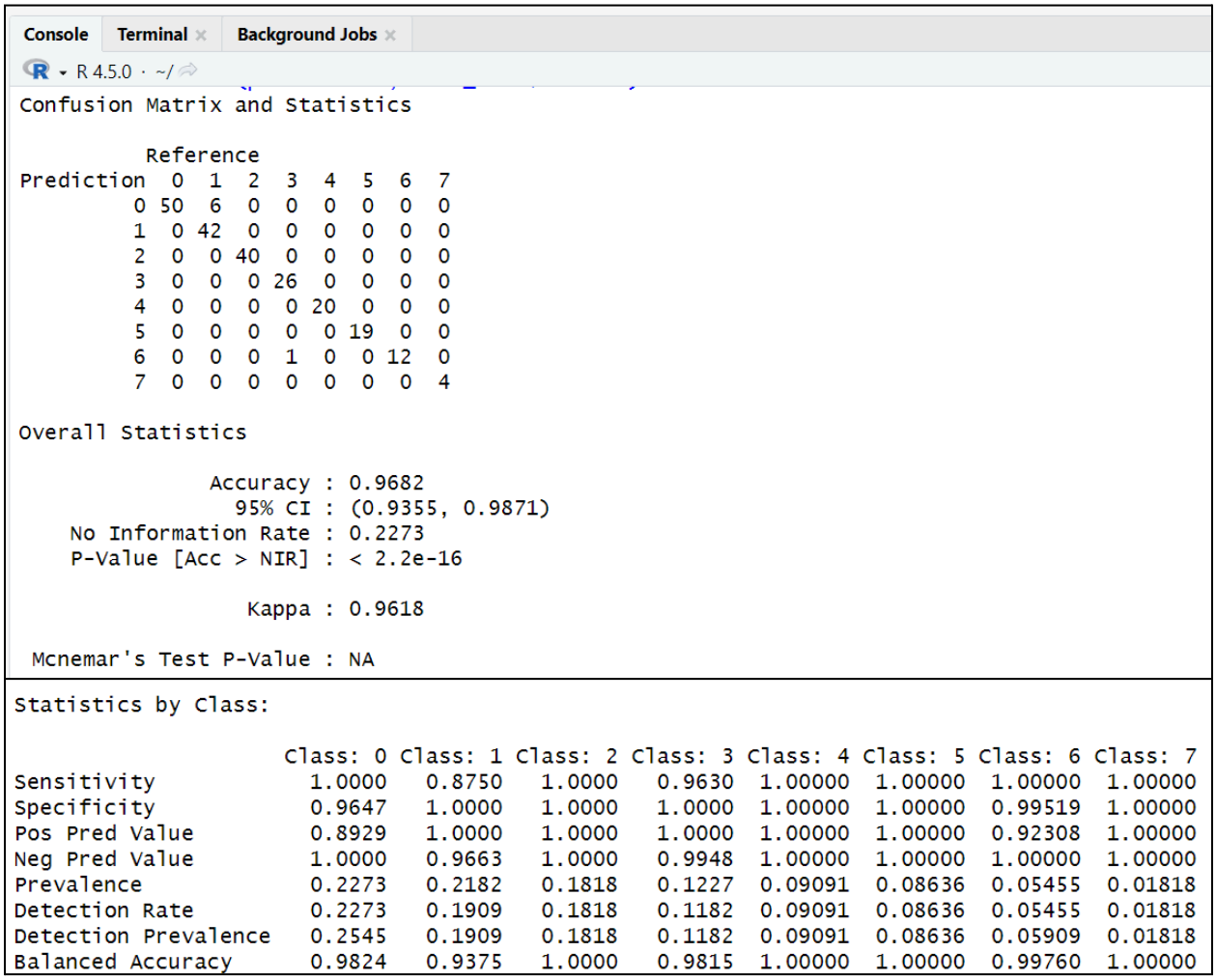
We'll use PCA-reduced features as input and cluster labels as target.

```
# Get PCA embeddings (reduced data)
pca_features <- Embeddings(seurat_obj, "pca")[, 1:10]  # First 10 PCs

# Cluster labels as target
cell_labels <- as.factor(Idents(seurat_obj))
```

2. Train a k-Nearest Neighbors Classifier

This tells how well kNN classifies the clusters based on PCA features.

```
library(caret)

# Create a data frame
ml_data <- data.frame(pca_features)
ml_data$Cluster <- cell_labels

# Split into training and testing
set.seed(123)
train_idx <- createDataPartition(ml_data$Cluster, p = 0.7, list = FALSE)
train_data <- ml_data[train_idx, ]
test_data <- ml_data[-train_idx, ]

# Train kNN model
knn_model <- train(Cluster ~ ., data = train_data, method = "knn", tuneLength = 5)

# Predict on test data
predictions <- predict(knn_model, newdata = test_data)

# Confusion matrix
confusionMatrix(predictions, test_data$Cluster)
```



```
Console   Terminal   Background Jobs
R 4.5.0 · ~/
Confusion Matrix and Statistics

          Reference
Prediction  0  1  2  3  4  5  6  7
         0 50  6  0  0  0  0  0  0
         1  0 42  0  0  0  0  0  0
         2  0  0 40  0  0  0  0  0
         3  0  0  0 26  0  0  0  0
         4  0  0  0  0 20  0  0  0
         5  0  0  0  0  0 19  0  0
         6  0  0  0  1  0  0 12  0
         7  0  0  0  0  0  0  0  4

Overall Statistics

               Accuracy : 0.9682
                 95% CI : (0.9355, 0.9871)
    No Information Rate : 0.2273
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.9618

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: 0 Class: 1 Class: 2 Class: 3 Class: 4 Class: 5 Class: 6 Class: 7
Sensitivity            1.0000   0.8750   1.0000   0.9630  1.00000  1.00000  1.00000  1.00000
Specificity            0.9647   1.0000   1.0000   1.0000  1.00000  1.00000  0.99519  1.00000
Pos Pred Value         0.8929   1.0000   1.0000   1.0000  1.00000  1.00000  0.92308  1.00000
Neg Pred Value         1.0000   0.9663   1.0000   0.9948  1.00000  1.00000  1.00000  1.00000
Prevalence             0.2273   0.2182   0.1818   0.1227  0.09091  0.08636  0.05455  0.01818
Detection Rate         0.2273   0.1909   0.1818   0.1182  0.09091  0.08636  0.05455  0.01818
Detection Prevalence   0.2545   0.1909   0.1818   0.1182  0.09091  0.08636  0.05909  0.01818
Balanced Accuracy      0.9824   0.9375   1.0000   0.9815  1.00000  1.00000  0.99760  1.00000
```

Evaluating the Model's Performance Based on Machine Learning Metrics

Confusion Matrix

This shows how many times the model correctly or incorrectly predicted each cluster (label/class).

```
Confusion Matrix and Statistics

          Reference
Prediction  0  1  2  3  4  5  6  7
         0 50  6  0  0  0  0  0  0
         1  0 42  0  0  0  0  0  0
         2  0  0 40  0  0  0  0  0
         3  0  0  0 26  0  0  0  0
         4  0  0  0  0 20  0  0  0
         5  0  0  0  0  0 19  0  0
         6  0  0  0  1  0  0 12  0
         7  0  0  0  0  0  0  0  4
```

- Each row = what the model predicted
- Each column = the actual true label

For example:

- The model predicted class 0 correctly 50 times ([0,0] = 50)
- It also mistakenly predicted 6 cells as class 0 that were actually class 1 ([0,1] = 6)

Overall Accuracy

```
Accuracy: 0.9682
```

This means the model correctly classified **96.82%** of the cells. That's **very high** and indicates the classifier is doing a great job.

95% CI (Confidence Interval)

```
(0.9355, 0.9871)
```

This range tells us that we're **95% confident** the true accuracy lies between **93.55% and 98.71%**. My model is **highly accurate**, with very few misclassifications. The only real mistake was with class 0 being confused with class 1 a few times. For all other clusters, the classifier is spot on.

Term	What it Means
Kappa (0.9618)	Adjusted agreement beyond chance — anything above 0.8 is excellent.
Sensitivity (Recall)	How well it identifies actual positive cases per class (e.g., class 0, 1, 2…).
Specificity	How well it avoids false positives — high = good.
Precision (Pos Pred Value)	Of the predicted class, how many were actually correct
Balanced Accuracy	Average of sensitivity and specificity — useful when classes are imbalanced.

Summary of Work Flow

In this project, we successfully analyzed single-cell RNA sequencing data using the Seurat package in R. Starting from raw 10x Genomics HDF5 data, we performed essential preprocessing steps, including normalization, identification of highly variable features, scaling, and dimensionality reduction using PCA. Through clustering and visualization with UMAP, we identified distinct cell populations. Differential gene expression analysis between these clusters revealed key marker genes that characterize different cell types. Finally, using a machine learning classifier (k-Nearest Neighbors), we built a predictive model to classify cells based on their gene expression profiles. The model achieved a high accuracy of 96.8%, demonstrating strong performance in distinguishing between cell types.

Results

This project provided hands-on experience with single-cell RNA sequencing data. Using Seurat, we successfully:

- Preprocessed and visualized the data
- Identified biologically meaningful clusters
- Detected marker genes
- Trained a machine learning classifier (k-Nearest Neighbors)
    - We built a predictive model to classify cells based on their gene expression profiles.
    - The model achieved a high accuracy of **96.8%**, demonstrating strong performance in distinguishing between cell types

These results demonstrate how combining single-cell genomics with machine learning helps uncover complex cellular identities and behaviors, essential for modern biomedical research.

Resources used:

1. Article: A practical guide to single-cell RNA-sequencing for biomedical research and clinical applications | Genome Medicine
2. Dataset: Datasets - 10x Genomics